

INTELIGENCIA ARTIFICIAL Y MINI ROBOTS

MACHINE LEARNING

Desarrollo de los ejercicios propuestos en la sección 6.12

Ejercicio 1. Multiplicación de Matrices 2×2 vs ML

Ejercicio 2. Support Vector Machine (SVM)

Ejercicio 3. K-Nearest Neighbors (KNN)

Ejercicio 4. Árboles de Decisión con Datos Gubernamentales

Presentado por:

Edinson Sanchez Fuentes (edsanchezf@unal.edu.co)

Brayan Alejandro Muñoz Pérez (bmunozp@unal.edu.co)

Joan Sebastián Roberto Puerto (jroberto@unal.edu.co)

Docente: José J. Martínez P.

Universidad Nacional de Colombia, Sede Bogotá

Octubre de 2026

Contenido

1. Introducción
 - 1.1 Metodología
2. Ejercicio 1. Multiplicación de Matrices 2×2 vs ML
 - 2.1 El experimento y conjunto de datos
 - 2.2 Diseño del modelo y comparación analítica
 - 2.3 Análisis de costo computacional
 - 2.4 Conclusiones
3. Ejercicio 2. Support Vector Machine (SVM)
 - 3.1 Documentación mejorada del algoritmo
 - 3.2 Diseño de la aplicación
4. Ejercicio 3. K-Nearest Neighbors (KNN)
 - 4.1 Documentación mejorada del algoritmo
 - 4.2 Diseño de la aplicación
5. Ejercicio 4. Árboles de Decisión (Datos Gubernamentales)
 - 5.1 Documentación mejorada del algoritmo
 - 5.2 Selección de los datos (Datos Abiertos Colombia)
 - 5.3 Diseño de la aplicación
6. Uso de herramientas de IA Generativa
 - 6.1 Herramientas empleadas
 - 6.2 Prompts utilizados
 - 6.3 Alcance y limitaciones observadas
7. Programas, repositorio y ejecución
 - 7.1 Estructura de los archivos
 - 7.2 Ejecución en Google COLAB / Jupyter
8. Bibliografía

1. Introducción

Este documento presenta el desarrollo de los ejercicios propuestos en la sección 6.12 del capítulo Introducción a Machine Learning. El objetivo es aplicar los conceptos de aprendizaje automático (tanto supervisado como no supervisado), evaluar su pertinencia frente a métodos analíticos tradicionales, y profundizar en la documentación y aplicación práctica de algoritmos clásicos como SVM, KNN y Árboles de Decisión, cumpliendo con la directriz de usar bases de datos del gobierno nacional.

1.1 Metodología

Para garantizar el rigor de las implementaciones y cumplir con los requisitos del curso, se definió la siguiente metodología:

- **Comparación de paradigmas:** En el Ejercicio 1 se contrasta la programación tradicional determinista con el enfoque de Machine Learning utilizando métricas de costo computacional (flops/tiempo de inferencia).
- **Líneas base y métricas:** Para los algoritmos de clasificación (SVM, KNN, Árboles), los modelos se evaluarán utilizando validación cruzada y la Matriz de Confusión (para obtener Precisión, Sensibilidad y Exactitud).
- **Selección de Datos Reales:** Se incorporó el uso de la plataforma de Datos Abiertos Colombia (Ley 1712 de 2014) para garantizar la aplicabilidad de los modelos a entornos del mundo real.

2. Ejercicio 1. Multiplicación de Matrices 2×2 vs ML

Enunciado: Considere la figura 6.1. Conforme un data set con resultados de 50.000

multiplicaciones de matrices 2×2 , generadas aleatoriamente con números enteros entre -20 y 20. Una vez entrenado el modelo, úselo con 10 ejemplos y compare los resultados con los que se producen al realizar la operación analíticamente. Haga un cálculo de qué es más costoso computacionalmente.

2.1 El experimento y conjunto de datos

Para este ejercicio, el conjunto de datos (dataset) se genera sintéticamente. Las características (features) de entrada son los 8 elementos numéricos correspondientes a dos matrices A y B de 2×2 . Los rótulos (labels) son los 4 elementos de la matriz resultante $C = A \times B$. Se generan 50.000 instancias aleatorias con valores discretos $x_i \in [-20, 20]$.

2.2 Diseño del modelo y comparación analítica

El problema se aborda como una regresión múltiple (4 salidas continuas). Se entrena un modelo (por ejemplo, una red neuronal densa simple o Regresión Lineal Múltiple) para mapear

los 8 valores de entrada a los 4 valores de salida.

Al pasar 10 ejemplos nuevos por el modelo, se obtienen predicciones que, aunque muy cercanas al valor real si el modelo converge bien, presentan un error residual estadístico (RMSE > 0), a diferencia del método analítico que garantiza un error de 0 de forma determinista.

2.3 Análisis de costo computacional

- **Método Analítico:** Para calcular la matriz $C = A \times B$ de 2×2 , se requiere realizar por definición 8 multiplicaciones de escalares y 4 sumas. El costo computacional analítico es $O(1)$ para matrices de tamaño fijo. Esto se traduce en unas pocas docenas de ciclos de reloj del procesador.
- **Método Machine Learning:**
 1. **Entrenamiento:** Procesar 50.000 ejemplos a través de múltiples épocas, calculando el gradiente descendente y actualizando pesos, requiere miles de millones de operaciones de punto flotante (FLOPS).
 2. **Inferencia:** Predecir un solo ejemplo implica multiplicar un vector de entrada de tamaño 8 por las matrices de pesos de las capas ocultas y aplicar funciones de activación. Solo la propagación hacia adelante requiere cientos de operaciones, excediendo por mucho las 12 operaciones matemáticas originales.

2.4 Conclusiones

1. Usar Machine Learning para resolver una operación aritmética determinista simple es altamente ineficiente y computacionalmente costoso.
2. Los modelos de ML aproximan funciones probabilísticas; por lo tanto, insertan un margen de error (ruido) innecesario en un cálculo que posee un algoritmo analítico exacto.
3. Este ejercicio demuestra que el aprendizaje de máquina está diseñado para problemas complejos, entornos fluctuantes y reconocimiento de patrones donde los algoritmos tradicionales fallan o son imposibles de codificar a mano, no para sustituir el álgebra lineal básica.

3. Ejercicio 2. Support Vector Machine (SVM)

Enunciado: Estudie el algoritmo SVM con todo detalle, mejore su documentación y con base en el haga cambios para una aplicación.

3.1 Documentación mejorada del algoritmo

Las Máquinas de Vectores de Soporte (SVM) son modelos de aprendizaje supervisado potentes y versátiles.

- **Principio de Maximización del Margen:** A diferencia de la regresión logística que simplemente traza una línea divisoria, SVM busca el hiperplano óptimo que no solo separa

las clases, sino que maximiza la distancia (margen) entre los puntos de datos más cercanos de ambas clases.

- **Vectores de Soporte:** Son las instancias críticas del dataset que se ubican en el borde del margen. El hiperplano depende matemáticamente *únicamente* de estos vectores.
- **El Kernel Trick:** En datasets no separables linealmente (como un anillo de datos rodeando a otro), SVM utiliza funciones Kernel (Polinomial, RBF - Radial Basis Function) para mapear las características a un espacio de mayor dimensión $\mathbb{R}^n \rightarrow \mathbb{R}^m$ (donde $m > n$). Esto permite que un hiperplano lineal en el espacio de alta dimensión funcione como una frontera de decisión compleja y curva en el espacio original.

3.2 Diseño de la aplicación

- **Dataset:** Dataset de Cáncer de Mama de Wisconsin (Breast Cancer Wisconsin).
- **Objetivo:** Clasificar si un tumor es benigno o maligno basado en 30 características geométricas celulares.
- **Implementación:** Se probará un SVM con Kernel Lineal vs Kernel RBF para evaluar cómo la no-linealidad afecta la Matriz de Confusión, buscando maximizar la *Sensibilidad* (reducir los falsos negativos en diagnósticos médicos).

4. Ejercicio 3. K-Nearest Neighbors (KNN)

Enunciado: Estudie el algoritmo de K- Nearest, con todo detalle mejore su documentación y con base en el haga cambios para una aplicación.

4.1 Documentación mejorada del algoritmo

El algoritmo de los K-Vecinos Más Cercanos es un método de aprendizaje supervisado no paramétrico y de tipo *Lazy Learning* (aprendizaje perezoso).

- **Aprendizaje Perezoso:** No existe una fase de entrenamiento donde se optimicen pesos (como en Redes Neuronales o SVM). El "entrenamiento" consiste simplemente en cargar el dataset en memoria. Todo el cálculo pesado ocurre en la fase de inferencia.
- **Métricas de Distancia:** Dependiendo del dominio, la similitud se mide geométricamente.

Las más usadas son la Euclidiana (L_2), Manhattan (L_1) o Minkowski.

- **Decisión por Votación:** Al introducir una nueva instancia, se calcula su distancia contra todos los puntos del dataset. Se extraen los K puntos con la menor distancia. La predicción es la moda (clase más frecuente) de estos vecinos. El hiperparámetro K controla el balance sesgo-varianza: un $K = 1$ sobreajusta capturando ruido, un K muy grande generaliza demasiado (subajuste).

4.2 Diseño de la aplicación

- **Dataset:** Reconocimiento de Vinos (Wine Dataset).

- **Objetivo:** Clasificar vinos en 3 cultivares diferentes basándose en 13 propiedades físicoquímicas (alcohol, ácido málico, ceniza, etc.).
- **Implementación:** Se requiere pre-procesamiento obligatorio (estandarización con StandardScaler), dado que KNN es altamente sensible a la escala de las magnitudes. Se realizará una curva de validación para encontrar el valor óptimo del hiperparámetro K .

5. Ejercicio 4. Árboles de Decisión (Datos Gubernamentales)

Enunciado: Estudie el algoritmo de árboles de decisión, con todo detalle mejore su documentación y con base en el haga cambios para una aplicación. (*En alguno de los problemas usar datos de alguna entidad gubernamental*).

5.1 Documentación mejorada del algoritmo

Los Árboles de Decisión construyen modelos en forma de diagrama de flujo, dividiendo el espacio de características mediante reglas lógicas de tipo "if-then-else".

- **Nodos y División (Splitting):** El algoritmo comienza en el nodo raíz y divide recursivamente los datos evaluando las características. Busca la característica y el punto de corte que maximice la "pureza" de los nodos resultantes.
- **Métricas de Impureza:** Se basan principalmente en el **Índice Gini** (probabilidad de clasificar incorrectamente una instancia aleatoria) o la **Entropía** (medida de la teoría de la información sobre el desorden del nodo).
- **Interpretabilidad y Sobreajuste:** Su gran ventaja es ser un modelo de "caja blanca" altamente interpretable. Su mayor desventaja es la alta propensión al sobreajuste si se deja crecer el árbol hasta la profundidad máxima, por lo que se requiere usar técnicas de pre-poda (limitar max_depth) o post-poda.

5.2 Selección de los datos (Datos Abiertos Colombia)

Cumpliendo con los requerimientos, se empleará un dataset del portal www.datos.gov.co.

- **Dataset Seleccionado:** Histórico de Accidentes de Tránsito o Reportes de Hurto por Modalidad (dependiendo de la disponibilidad de la API de Socrata).
- **Objetivo:** Clasificar el nivel de severidad o la clase de un incidente de tránsito (ej. Daños vs Heridos/Muertos) utilizando características como el día de la semana, comuna, rango de horas, y tipo de vía.

5.3 Diseño de la aplicación

Se extraerán los datos mediante la API de Socrata con pandas. Se realizará una codificación (One-Hot Encoding) de las variables categóricas. Se entrenará el árbol limitando su profundidad (max_depth=4) para poder graficarlo y extraer las reglas lógicas (ej: "Si es fin de semana y ocurre en la madrugada, la probabilidad de accidentes con heridos aumenta

significativamente").

6. Uso de herramientas de IA Generativa

De acuerdo a los parámetros del curso, a continuación se documenta el uso de herramientas de IA:

Herramienta	Uso que se le dio	Utilidad observada
Asistente de IA (ChatGPT/Gemini)	Estructuración del documento Markdown, adaptación al formato del reporte y explicaciones teóricas de algoritmos.	Aceleró la redacción del documento y aseguró la consistencia en el formato de presentación requerido.
Documentación Scikit-Learn	Consulta de hiperparámetros para SVM, KNN y Decision Trees, y API de métricas de confusión.	Indispensable para asegurar el uso correcto de StandardScaler y los parámetros kernel y max_depth.
Portal Datos Abiertos Colombia	Exploración y consulta de conjuntos de datos gubernamentales a través de API (Socrata).	Permitió la integración de un caso de uso real para el ejercicio 4.

6.1 Prompts utilizados

- "Estructura un informe formal en Markdown para un trabajo universitario usando el siguiente texto base, imitando la estructura de un reporte de laboratorio en formato de la Universidad Nacional."
- "Genera una justificación analítica en notación Big-O comparando la multiplicación tradicional de matrices 2×2 frente al uso de descenso de gradiente en Machine Learning."

6.2 Alcance y limitaciones observadas

- El asistente es muy útil para dar formato y estructurar la base matemática (LaTeX).
- La IA es propensa a inventar datasets si no se le restringe. Hubo que especificar explícitamente el diseño usando Datos Abiertos Colombia para asegurar que se cumplieran los lineamientos exactos del curso.

7. Programas, repositorio y ejecución

7.1 Estructura de los archivos

El proyecto consta de este documento principal y los scripts de los ejercicios en formato Notebook:

```
6. Machine Learning/
|— README.md
|— requirements.txt
|— Informe_Ejercicios.pdf (Este documento)
|— scripts/
|   |— Ejercicio_1_Matrices.py
|   |— Ejercicio_2_SVM.py
|   |— Ejercicio_3_KNN.py
|   |— Ejercicio_4_Arboles_Decision.py
```

8. Bibliografía

- [1] Martínez, J. J. (2026). *Introducción a Machine Learning*, capítulo 6. Material del curso Inteligencia Artificial y Minirobots, Universidad Nacional de Colombia.
- [2] Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- [3] Portal de Datos Abiertos de Colombia. Ley 1712 de 2014. <https://www.datos.gov.co>
- [4] Scikit-learn: Machine Learning in Python. Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.